GRAVITY FLIP — Combined Documentation

Daily Game Development Series, Day 3 — Whitepaper · History · Source Code Detail

Project	GRAVITY FLIP
Series	Daily Local Game Development Series — Day 3
Author	Mayank Swaraj
Brand	Mayank Creations
Stack	Vanilla JavaScript, HTML5 Canvas, Web Audio API
Document Type	Combined Whitepaper + History + Source Code Detail

■ WHITEPAPER

1. Overview

GRAVITY FLIP is Day 3 of the daily vanilla JavaScript game development series under Mayank Creations. It is a side-scrolling auto-runner built on a single mechanic: tapping flips the direction of gravity, sending the player entity between the upper and lower halves of a bounded track to navigate gaps in procedurally placed walls.

Mechanically, Day 1 was a turn-based grid puzzle and Day 2 was a real-time vertical dodge game. Day 3 introduces continuous horizontal motion, a one-input control scheme, and a score-multiplier system tied to optional collectibles — a distinct genre from both prior entries.

2. Design Goals

- One-input accessibility: the entire game is playable with a single repeated tap, making it equally suited to touch, mouse click, and keyboard spacebar without any separate control mapping.
- Procedural infinite generation: walls and gems are spawned at runtime using a rolling spawn point, removing the need for hand-authored level data.
- Multiplier incentive layer: gems are optional but reward consistent collection with a score multiplier up to x8, adding a risk-reward dimension on top of the pure survival objective.

- Continuous difficulty ramp: wall speed and spawn rate tighten as elapsed time grows, implemented as continuous functions rather than discrete tier thresholds.

3. Core Mechanics

The player entity has a vertical position and velocity. On each frame, gravity — a signed constant whose sign flips on every player input — is added to velocity, and velocity is applied to position. Velocity is clamped to prevent runaway acceleration. The track has a hard ceiling and floor; touching either kills the run instantly.

Walls are represented as gap objects: a gap top coordinate, a gap bottom coordinate, and a full-width solid block above and below the gap. The player must pass through the gap. Collision is resolved with axis-aligned bounding box checks against both blocks. Walls scroll from right to left at a speed that increases continuously with elapsed time.

Gems spawn at the horizontal and vertical midpoint of a wall gap and are collected by proximity. Each gem collected increments the score multiplier by one up to a cap of eight; the multiplier applies to both wall-clear score bonuses and passive survival score accumulation.

4. Rendering

All rendering is done on an HTML5 canvas using the 2D context API. The player is drawn as a diamond polygon with a gradient fill and a glow shadow. Walls use a gradient-filled rectangle pair with a green-tinted neon stroke. Gems are filled arcs with glow. Track boundaries are filled solid blocks. A flip animation is applied to the player by rotating the canvas context around the player's center for the brief duration of a gravity switch, giving visual confirmation of the input.

5. Audio Design

- Flip: short sine tone at 280 Hz, communicates the input event without being fatiguing on repeated taps.
- Gem collect: bright sine tone at 740 Hz, distinct from the flip tone so both can occur close together without ambiguity.
- Hit/death: immediate sawtooth burst followed after a short delay by a three-note descending sequence.
- Same AudioContext lazy-init and first-gesture unlock pattern as Days 1 and 2.

6. Outcome

Single HTML file, no external dependencies, runs offline. Touch-tap and SPACE/click input with no separate mobile build required — the one-input scheme is inherently device-agnostic.

■ HISTORY PAPER

1. Position in the Series

GRAVITY FLIP is the third consecutive daily entry, following CRATE PUSH (grid puzzle) and NEON DODGER (canvas dodge). The series brief specifies a completely different game each day. Day 3 was chosen to introduce: horizontal rather than vertical player motion, a one-button input model, and procedural generation replacing both hand-authored levels and random top-down spawning.

2. Build Timeline

Day 3 followed the consolidated single-pass model established on Day 2: game logic, rendering, audio, and mobile input were all implemented together without a separate mobile retrofit stage. The one-input scheme (tap/click/space) meant touch support was inherent from the initial design rather than an add-on.

Documentation was produced immediately as a combined PDF, consistent with the format established after Day 1 and confirmed again on Day 2.

3. Decisions and Trade-offs

- Continuous scroll over discrete screens: the infinite runner model suits the single-tap mechanic better than screen-by-screen level transitions.
- Gap-based walls over free-form obstacles: guarantees every generated configuration is passable, avoiding unwinnable procedural layouts without requiring a solvability check.
- Velocity partial-reverse on flip: multiplying vertical velocity by -0.4 rather than zeroing it on a gravity flip retains a small amount of the prior momentum, which makes flip timing feel weighty rather than instant-response.
- Multiplier cap at x8: high enough to matter as a goal, low enough that a single gem miss does not feel catastrophic to an experienced player.

4. Divergence from Prior Entries

- Day 1 (CRATE PUSH): grid-based, turn-based, hand-authored levels, DOM rendering.
- Day 2 (NEON DODGER): canvas, real-time, top-down, drag positioning control.
- Day 3 (GRAVITY FLIP): canvas, real-time, side-scrolling, one-input flip control, procedural generation.

■ SOURCE CODE DETAIL

1. Gravity and Physics

Gravity is a signed multiplier (1 or -1) applied per frame. Flipping negates the sign and partially reverses current vertical velocity. Velocity is clamped symmetrically to prevent degenerate acceleration over long runs.

```
grav *= -1;
p.vy *= -0.4;

// Per frame:
var GRAVITY = 0.022 * grav;
p.vy += GRAVITY * dt;
p.vy = Math.max(-14, Math.min(14, p.vy));
p.y += p.vy * (dt / 16);
```

2. Procedural Wall Generation

A rolling spawn point accumulates horizontal distance each frame. When it exceeds a threshold derived from a shrinking interval, a wall is appended and the spawn point resets. Gap top coordinate is randomized within track bounds minus gap height to guarantee the gap is always reachable.

```
function spawnWall() {
var gapTop = TRACK_TOP + Math.random() * (TRACK_BOT - TRACK_TOP - GAP_H);
walls.push({ x: spawnX, gapTop: gapTop, gapBot: gapTop + GAP_H, scored: false });
if (Math.random() < 0.55)
gems.push({ x: spawnX + WALL_W + 40, y: gapTop + GAP_H / 2, r: 9, alive: true });
}
```

3. Difficulty Scaling

Speed and spawn interval are continuous functions of elapsed time in milliseconds, clamped to a maximum speed to prevent the game becoming physically unplayable.

```
function speed() {
return Math.min(SPEED_MAX, SPEED_BASE + elapsed / 9000);
}
var interval = Math.max(210, 420 - elapsed / 120);
```

4. Collision Detection

Player vs wall uses AABB overlap checked against both the top block (TRACK_TOP to gapTop) and the bottom block (gapBot to TRACK_BOT). Track boundary collision is a simple coordinate range check. Gem collection uses a Euclidean distance check between player center and gem center.

5. Player Flip Animation

A flipping timer is set on each gravity change. While the timer is positive, the canvas context is rotated around the player center by an angle proportional to remaining timer time and the current gravity direction, giving a brief spin that confirms the flip input visually without affecting hitbox.

```
if (p.flipping > 0) {
angle = Math.PI * (1 - p.flipping / 120) * grav;
}
cx.save();
cx.translate(PLAYER_X, p.y);
cx.rotate(angle);
// draw diamond shape
cx.restore();
```

6. Known Constraints

- Best score is session-only; no localStorage persistence in this build.
- No pooling on wall/gem arrays; splice is used directly, adequate at these object counts.
- dt is clamped to 50ms per frame to prevent large position jumps after tab-background pauses.

"Every system I build is a brick in a larger, self-taught architecture of independent AI work."

- Mayank Swaraj